

Framing a Spatiotemporal Burn-State Transition Model

1 Problem Overview

We consider a landscape represented as a raster grid observed daily. Each pixel has one of five possible burn states:

- OU: outside unburned
- IH: intense heat
- SH: scattered heat
- OH: isolated heat
- IU: interior unburned

The objective is to model the burn state of each pixel at day t as a function of its own previous burn states, the previous burn states of its eight neighboring pixels, static environmental variables such as fuels and topography, and recent weather or fire-weather conditions.

Let

$$Y_{i,t} \in \{OU, IH, SH, OH, IU\}$$

denote the burn state of pixel i at day t . Let $N(i)$ denote the eight-cell neighborhood around pixel i , and let k denote the number of lag days used as history. A general modeling formulation is:

$$P(Y_{i,t} \mid Y_{i,t-1:t-k}, Y_{N(i),t-1:t-k}, X_i, W_{i,t-1:t-k}),$$

where X_i represents static environmental covariates and $W_{i,t-1:t-k}$ represents lagged weather or fire-weather variables.

2 Ten-Step Framing

Step 1: Define the Problem as a Spatiotemporal State-Transition Model

Frame the problem as a supervised spatiotemporal classification problem on a raster lattice. Each pixel is a cell, each day is a time step, and the response variable is the burn state of the pixel at the next day.

The daily update rule can be written conceptually as:

$$\text{state tomorrow} = f(\text{recent focal-pixel state history}, \text{recent neighborhood state history}, \text{environment}, \text{weather}).$$

This is similar to a probabilistic cellular automaton, except that the transition rule is learned from data rather than manually specified.

Step 2: Define the Prediction Target

There are two main choices for the response variable.

Direct state prediction. Predict the burn state at day t :

$$Y_{i,t}.$$

This creates a five-class classification problem with classes OU, IH, SH, OH, and IU.

State-transition prediction. Predict the transition from the previous day to the current day:

$$Y_{i,t-1} \rightarrow Y_{i,t}.$$

This framing is often more interpretable because fire behavior is fundamentally about state transitions, such as $OU \rightarrow IH$ or $IH \rightarrow IU$.

Step 3: Represent the Local Spatiotemporal Window

For each pixel-day, extract a local space-time cube containing the focal pixel and its eight neighbors over the previous k days. For example, using a 3×3 neighborhood and five lag days gives a local input window of size:

$$3 \times 3 \times 5.$$

Because burn state is categorical, states can be encoded as integer labels or one-hot vectors. A one-hot encoding is often preferable for machine-learning models:

$$OU = [1, 0, 0, 0, 0], \quad IH = [0, 1, 0, 0, 0], \quad SH = [0, 0, 1, 0, 0],$$

$$OH = [0, 0, 0, 1, 0], \quad IU = [0, 0, 0, 0, 1].$$

Step 4: Organize Predictors into Three Groups

The predictors can be organized into three groups.

Burn-state history. Include the previous states of the focal pixel and its neighboring pixels over the lag window. Useful derived features include the number of IH neighbors, the number of SH neighbors, whether any neighbor was burning, and days since a nearby pixel burned.

Static environmental variables. Include spatial covariates such as fuel type, vegetation, elevation, slope, aspect, canopy cover, land cover, distance to roads or firelines, and other landscape variables.

Weather and fire-weather variables. Include lagged or summarized weather variables such as temperature, relative humidity, wind speed, wind direction, precipitation, vapor pressure deficit, energy release component, and fuel moisture. Useful summaries include mean conditions over the past three days, maximum wind speed over the lag window, minimum relative humidity, and cumulative precipitation.

Step 5: Begin with a Simple Markov Baseline

Start with a simple transition model based only on the previous state of the focal pixel:

$$P(Y_{i,t} \mid Y_{i,t-1}).$$

This produces a basic transition matrix showing how likely each state is to persist or change to another state.

Table 1: Example structure of a burn-state transition matrix.

From / To	OU	IH	SH	OH	IU
OU					
IH					
SH					
OH					
IU					

This baseline is useful because it provides a simple benchmark for all more complex models.

Step 6: Add Neighborhood Information

Next, include the states of the eight adjacent pixels. A useful formulation is:

$$P(Y_{i,t} \mid Y_{i,t-1}, Y_{N(i),t-1}).$$

Instead of including every neighbor state directly, it may be useful to summarize the neighborhood with count-based features, such as:

- number of IH neighbors at day $t - 1$,
- number of SH neighbors at day $t - 1$,
- number of OH neighbors at day $t - 1$,
- number of IU neighbors at day $t - 1$,
- whether any neighbor was in an active heat state,
- fraction of neighbors in active heat states.

Step 7: Add Lagged Neighborhood History

Extend the model to include several previous days of focal-pixel and neighborhood history:

$$P(Y_{i,t} \mid Y_{i,t-1:t-k}, Y_{N(i),t-1:t-k}).$$

Useful lagged features include the maximum number of IH neighbors over the last three days, the mean number of active-heat neighbors over the previous five days, and the number of days since the nearest neighboring heat detection.

Step 8: Add Fuels, Topography, and Weather

After establishing a state-history and neighborhood baseline, include environmental and weather covariates:

$$P(Y_{i,t} \mid Y_{i,t-1:t-k}, Y_{N(i),t-1:t-k}, X_i, W_{i,t-1:t-k}).$$

At this stage, the problem becomes a full spatiotemporal supervised learning problem. Weather variables should be aligned carefully with the burn-state time steps. Wind direction may be especially important because an upwind burning neighbor is not equivalent to a downwind burning neighbor.

Step 9: Choose an Initial Model Class

A practical first modeling approach is a tabular multiclass classifier using engineered features. Suitable initial models include:

- multinomial logistic regression,
- random forest,
- gradient boosted trees,
- XGBoost, LightGBM, or CatBoost.

These models allow rapid prototyping and provide interpretable feature importance. More complex models can be considered later, including convolutional neural networks, ConvLSTM models, temporal convolutional networks, U-Net-style raster-to-raster models, or graph neural networks.

Step 10: Validate with Spatial and Temporal Holdouts

Avoid random splitting of pixel-days because nearby pixels and adjacent dates are strongly auto-correlated. Random splits can produce overly optimistic accuracy.

Better validation strategies include:

- holding out entire fires,
- holding out later time periods,
- holding out spatial blocks,
- holding out fire-year combinations.

Evaluation should include per-class precision and recall, confusion matrices, transition-specific accuracy, and visual comparison between observed and predicted burn-state maps.

3 Recommended Starting Formulation

A recommended initial formulation is:

$$Y_{i,t} = f(Y_{i,t-1:t-3}, Y_{N(i),t-1:t-3}, X_i, W_{i,t-1:t-3}).$$

A practical implementation would use a pixel-day table where each row represents one pixel on one day. The target is either the burn state at day t or the transition from day $t - 1$ to day t .

Example Feature Table

Table 2: Example pixel-day training table.

Pixel	Day	State History	Neighborhood Features	Environment/Weather	Target
i	t	$Y_{i,t-1:t-3}$	counts by state	fuel, slope, wind, RH	$Y_{i,t}$

This formulation provides an interpretable and defensible baseline before moving to more complex deep-learning or raster-to-raster approaches.